
۱. کلاس (Class)

تعریف کاربردی:

کلاس طرح یا قالبی است که ویژگی‌ها (متغیرها) و رفتارها (متدها) را برای اشیاء تعریف می‌کند. شما می‌توانید از روی یک کلاس، چندین شیء ایجاد کنید. مثل نقشه ساخت یک ماشین.

کاربرد:

در پروژه‌هایی مثل مدیریت فروشگاه، کلاس‌هایی مانند Product, Customer, Order تعریف می‌شوند که هرکدام ویژگی و رفتار خاصی دارند.

مثال ساده:

```
public class Product {  
  
    String name;  
  
    double price;  
  
    void showInfo() {  
  
        System.out.println(name + " costs " + price + " USD");  
  
    }  
  
}
```

۲. شیء‌گرایی (Object-Oriented Programming)

تعریف کاربردی:

OOP یعنی ما برنامه را به مجموعه‌ای از اشیاء تقسیم می‌کنیم که هر کدام وظیفه‌ای مشخص دارند. این روش باعث سازمان‌یافتگی، انعطاف‌پذیری و قابلیت استفاده مجدد می‌شود.

چه مشکلی را حل می‌کند؟

برنامه‌های پیچیده را قابل کنترل می‌کند. بجای تکرار کد، کلاس‌ها و اشیاء با تعامل کار را انجام می‌دهند.

مثال کاربردی:

```
public class Main {  
  
    public static void main(String[] args) {
```

```
Product p = new Product();

p.name = "Camera";

p.price = 1500;

p.showInfo(); // خروجی: Camera costs 1500.0 USD

}

}
```

۳. ارث‌بری (Inheritance)

تعریف کاربردی:

کلاسی می‌تواند ویژگی‌ها و رفتارهای کلاس دیگر را به ارث ببرد. مثل اینکه یک Employee می‌تواند تمام ویژگی‌های یک Person را داشته باشد، و چیزهای مخصوص به خودش را هم اضافه کند.

کاربرد:

جلوگیری از تکرار کد و مدل‌سازی بهتر مفاهیم.

مثال ساده:

```
class Person {

    String name;

    void greet() {

        System.out.println("Hello, I'm " + name);

    }

}
```

```
class Employee extends Person {

    String jobTitle;

}
```

```
Employee e = new Employee();
```

```
e.name = "Ali";  
  
e.jobTitle = "Developer";  
  
e.greet(); // خروجی: Hello, I'm Ali
```

۴. کپسوله‌سازی (Encapsulation)

تعریف کاربردی:

یعنی مخفی کردن جزئیات داخلی یک کلاس و دسترسی به آن‌ها فقط از طریق متدها. به این صورت نمی‌توان مستقیماً مقادیر حساس را تغییر داد.

کاربرد:

امنیت داده‌ها، کنترل دقیق روی آنچه قابل تغییر است.

مثال ساده:

```
public class BankAccount {  
  
    private double balance; // فقط داخل کلاس قابل دسترسی است  
  
    public void deposit(double amount) {  
        if (amount > 0) balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

۵. اینترفیس (Interface)

تعریف کاربردی:

یک اینترفیس فقط امضای متدها را تعریف می‌کند، بدون پیاده‌سازی. کلاس‌هایی که اینترفیس را پیاده‌سازی می‌کنند، متعهد می‌شوند آن متدها را بسازند.

کاربرد:

ایجاد معماری انعطاف‌پذیر و جدا کردن "چی" از "چطور".

مثال:

```
interface Printable {
```

```
    void print();
```

```
}
```

```
class Report implements Printable {
```

```
    public void print() {
```

```
        System.out.println("Printing report...");
```

```
    }
```

```
}
```

۶. چندریختی (Polymorphism)

تعریف کاربردی:

یک شیء می‌تواند رفتاری متفاوت داشته باشد حتی اگر با همان نوع داده تعریف شده باشد. مثلاً متدی که با Printable کار می‌کند، می‌تواند با Report یا Invoice کار کند.

کاربرد:

کدهای عمومی‌تر و قابل توسعه‌تری نوشته می‌شود.

مثال:

```
Printable doc = new Report(); // Report است ولی رفتار Printable نوع
```

```
doc.print(); // خروجی: Printing report...
```

جمع‌بندی

کاربرد واقعی	فایده اصلی	مفهوم
تعریف موجودیت‌ها مثل User و Product	ساختاردهی و مدل‌سازی	Class
تقسیم سیستم به اشیاء قابل کنترل	مدیریت بهتر سیستم‌های بزرگ	OOP
ساخت کلاس‌های تخصصی مثل Admin از User	جلوگیری از تکرار و گسترش راحت	Inheritance
محافظت از موجودی حساب بانکی	امنیت داده و کنترل بهتر	Encapsulation
ارتباط ماژول‌ها با قرارداد مشترک	انعطاف‌پذیری و جداسازی رفتار	Interface
یک تابع برای چاپ هر نوع سند	نوشتن کدهای عمومی و قابل توسعه	Polymorphism